

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

1. Introduction:

This document introduces Homie, an AI-native rental assistant designed to solve the fragmented and inefficient rental search process in Malaysia. Current rental platforms such as iBilik, iProperty, and Facebook groups present unstructured and inconsistent data, forcing users to manually search, compare, and communicate with landlords. Homie addresses this by acting as an AI agent that automates the rental workflow, from data aggregation to decision-making and outreach.

1.1. Purpose:

*This document outlines the technical scope, architectural design and logic behind the **Homie** platform. It serves as a blueprint for the dual-level ReAct (Reason + Act) orchestration that manages multi-source scraping, multilingual normalization, and deterministic scoring.*

The key elements of the system that have been covered are as follows:

1. Architecture:

- a. *Client-Server with FastAPI backend, Next.js frontend, and cloud-hosted PostgreSQL. GLM-5.1 integration as core orchestration service layer.*

2. Data Flows:

- a. *Multi-stage pipeline: Validate Filters → Scrape Listings → Normalize Data → Score Results → Generate Report → Prepare Outreach. Data persisted in PostgreSQL with SSE streaming for real-time progress.*

3. Model Process:

- a. *End-to-end workflow demonstrating core features: search filter submission, multi-source scraping, intelligent listing normalization, deterministic scoring, and Telegram-based outreach handoff.*

4. Role of Reference:

- a. *Enables high-level alignment across distributed teams on architecture decisions, technology choices, and integration patterns. Serves as baseline for handoff documentation.*

1.2. **Background:**

Rental searching in Malaysia is currently fragmented across multiple platforms such as iProperty, iBilik, and Facebook groups. Users are required to manually browse listings, compare options, and contact landlords individually. This results in inefficiency, inconsistent data interpretation, and a time-consuming decision-making process.

1. *Previous Version:*

- a. Traditional search methods are manual and repetitive; users must visit each portal individually and manually compare listings with no shared ranking logic.*

2. *Changes in Major Architectural Components*

- a. Homie introduces a unified layer that automates discovery and initiates landlord outreach via Telegram.*

3. *New capabilities that are only introduced in this*

- a. Dual-Level ReAct Agentic Orchestration: Unlike static pipelines, Homie uses a top-level orchestrator to decide the workflow shape dynamically, coupled with per-stage sub-agents for specialized reasoning like extraction and translation.*
- b. Cross-Platform Multilingual Normalization: The system can ingest listings in Bahasa Malaysia, Chinese, or English and normalize them into a unified English schema using AI-powered field extraction.*
- c. Agentic Telegram Outreach: It utilizes the MTPROTO API to manage multi-turn conversations with landlords, automatically verifying "must-have" requirements and tracking conversation states.*
- d. Explainable 8-Dimension Scoring: Every listing is ranked using a deterministic 0–100 algorithm across eight specific criteria (Price, Transport, etc.), accompanied by plain-language AI explanations.*
- e. Live Workflow Transparency (SSE): The backend streams granular events via Server-Sent Events (SSE), allowing users to watch the AI "think" and execute each stage in real-time.*

- f. *Authenticated Facebook Scraping: Includes a guided login flow to access high-value rental listings within Facebook Groups and Marketplace that are typically hidden behind login walls..*

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Renter (End User)	Submits preferences via chat/form; reviews ranked shortlists.	Fast, relevant shortlists with clear explanations of why a listing fits, and automated outreach.
Property Agents/Landlords	Receive structured inquiries from potential tenants.	Professional, pre-verified inquiries that match their property criteria.

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

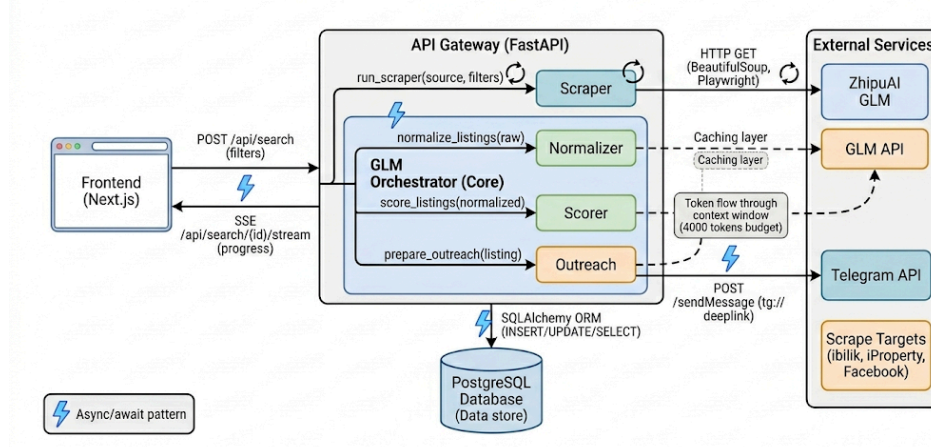
Type	Details
System	Web-based Dashboard (Next.js 14 / React).
Architecture	Asynchronous Backend (FastAPI) with a ReAct Orchestrator.

2.1.2 LLM as Service Layer

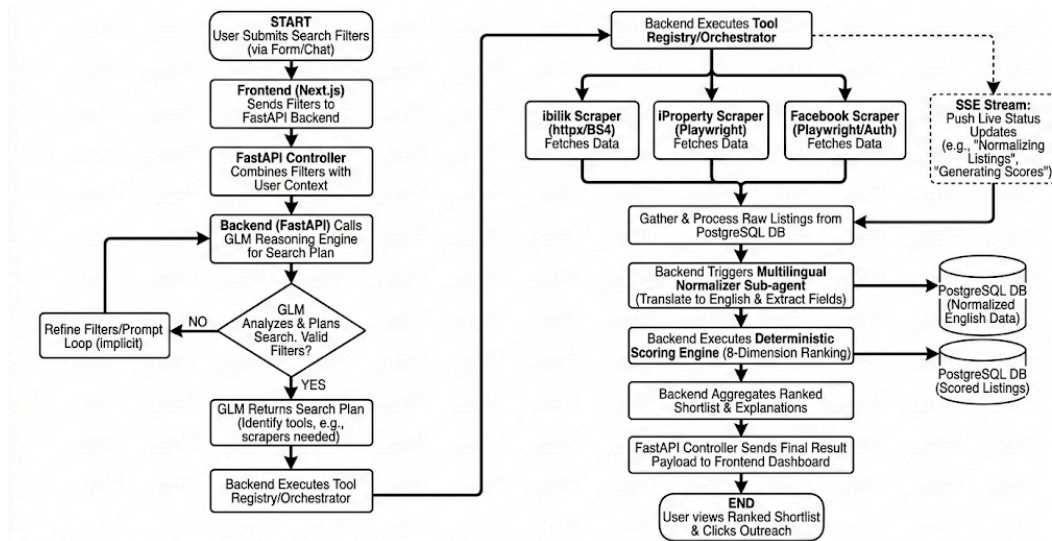
The GLM-5.1 model is integrated as a reasoning engine, not a simple chatbot:

- g. **Orchestration:** Decisions on tool selection (Scraping -> Normalizing -> Scoring) are made by the model based on session state.
- h. **Context Management:** Token limits are managed by chunking inputs and capping orchestrator iterations at 30.
- i. **Error Resilience:** The system includes deterministic fallbacks for explanations and reports if the AI layer fails.

2.1.3 Dependency Diagram



2.2.2. Sequence Diagram



- Search Submission:** User enters filters through the Next.js form or chat agent.
- Orchestration Initiation:** FastAPI triggers the GLM Orchestrator to plan the search.
- Parallel Scraping:** The system calls individual scraper modules (Playwright/BS4) for multiple sources concurrently.
- Normalization:** Raw, multilingual results are passed to the Normalization Sub-agent for translation and field mapping.
- Deterministic Scoring:** The backend engine applies the 8-dimension weights to generate a final score for each listing.
- Progress Streaming:** The backend pushes status updates to the frontend via SSE.
- Inquiry Generation:** On selection, the GLM generates a tailored inquiry draft for the landlord.

2.2. Technological Stack

2.2.1. Frontend:

- Next.js 14-React framework
- Tailwind CSS v4
- shadcn/ui-component library
- Type Script
- Native EventSource API — SSE client for live pipeline progress

2.2.2. Backend:

- Python + FastAPI — REST API server
- SQLAlchemy + PostgreSQL — primary data store (SQLite for local dev)
- Playwright (headless Chromium) — browser automation for iProperty, PropertyGuru, Facebook
- httpx + BeautifulSoup — HTTP scraping for ibilik
- Telethon (MTProto) — Telegram automated outreach client
- uv — Python package/env manager

2.2.3. AI / Orchestration

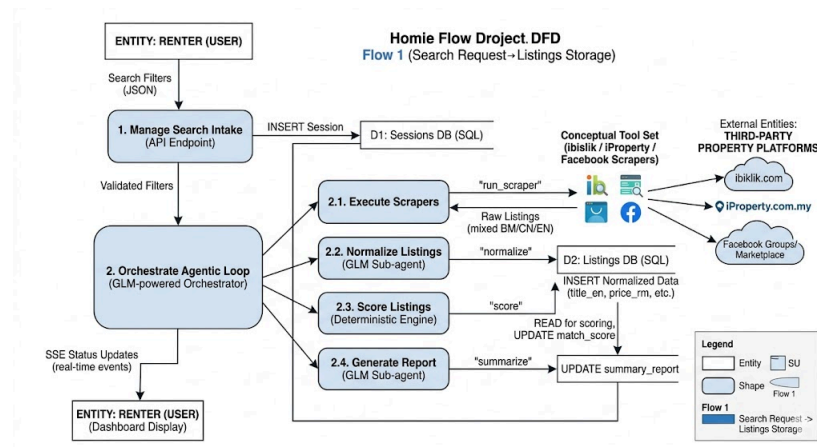
- GLM (ZhipuAI via ilmu.ai proxy, OpenAI-compatible API) — LLM backbone
- Dual-level ReAct pattern — top-level orchestrator + per-stage sub-agents
- Custom tool-calling loop — 9 orchestrator tools, 10-iteration sub-agent cap

2.3. Key Data Flows

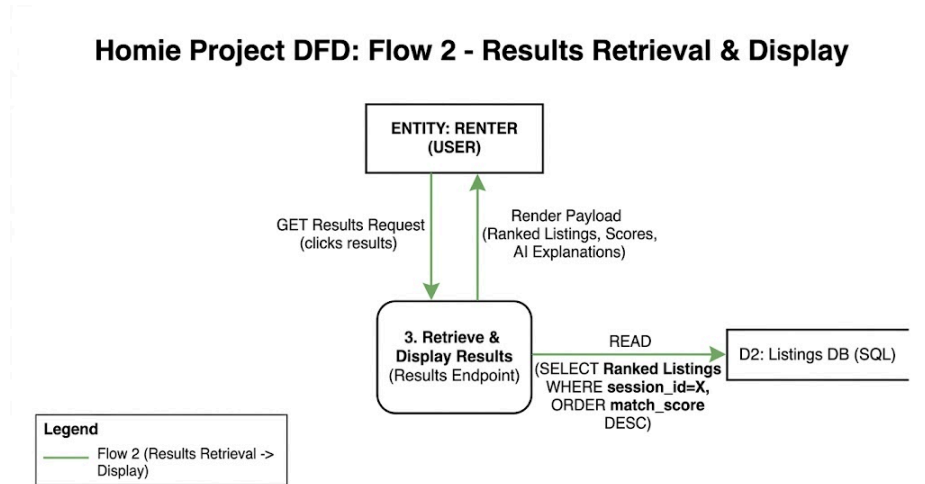
2.3.1. Data moves from the User Interface to the Orchestrator, which pulls raw data from External Listing Sources. This data is cleaned by the Normalization Agent and saved to the DB.

2.3.1.1. Data Flow Diagram (DFD)

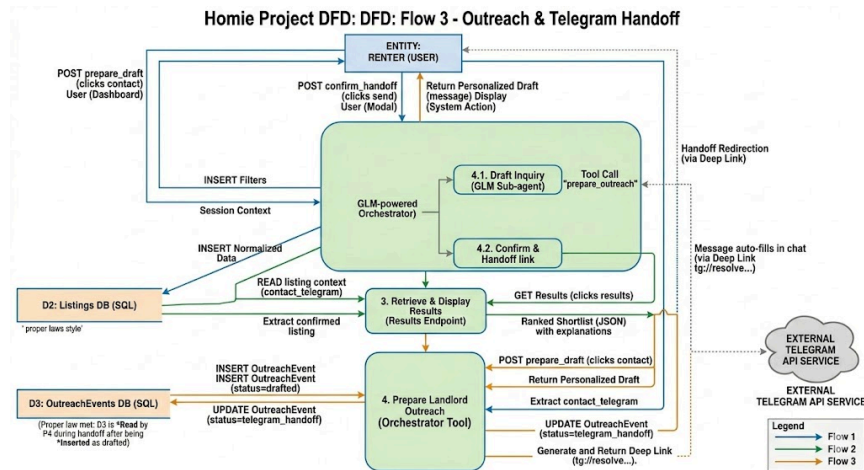
a. Homie DFD - Flow 1: Search Request → Listings Storage



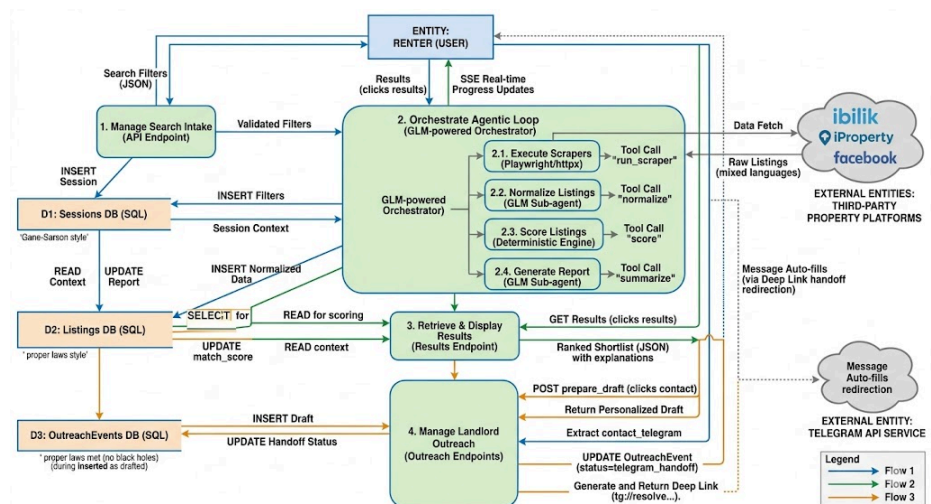
b. Homie DFD - Flow 2: Results Retrieval → Display



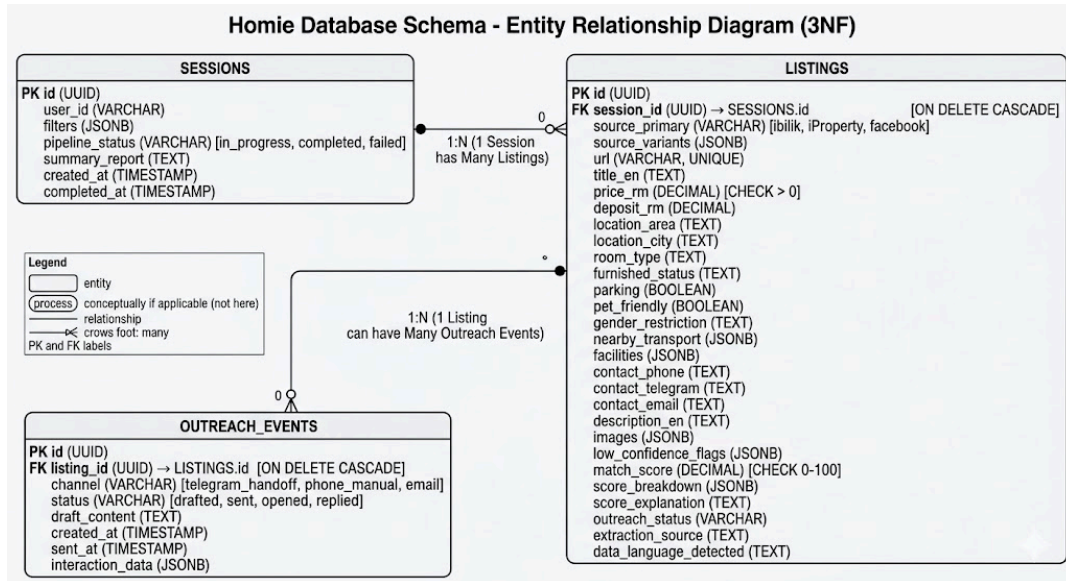
c. Homie DFD - Flow 3: Outreach → Telegram Handoff



d. Overall Data Flow



2.3.2. Normalized Database Schema



3. Functional Requirements & Scope

3.1. Minimum Viable Product:

#	Features	Description
1	AI Orchestration	GLM-driven task planning across the workflow stages.
2	Multi-Source Scraping	Concurrent collection from 3 major Malaysian platform which include iBilik, iProperty, and Facebook simultaneously.
3	Intelligent Listing Scoring	Listings are ranked by match to user preferences using 8-dimensional scoring + LLM explanations.
4	AI-Drafted Auto Outreach Messages	GLM generates personalized landlord messages based on user context + listing details. User confirms drafted message, Telegram app opens with auto-filled message to landlord.

3.2. Non-Functional Requirements (NFRs)

Quality (e.g.)	Requirements	Implementation
Scalability	The system must handle 100 concurrent users during peak hours	Use async FastAPI, connection pooling for PostgreSQL, and CloudRun auto-scaling.
Reliability	Search pipeline must complete with 95%+ success rate; partial failures handled gracefully	Retry scraper with exponential backoff; fallback to cached data if GLM unavailable; detailed error logging.

Maintainability	Code must be modular, well-documented, and testable.	Separate concerns (scrapers, normalizers, scorers as independent modules); type hints; docstrings
Token Latency	GLM sub-agents should respond within acceptable timeframes.	Prompt compression, response caching, input chunking
Cost Efficiency	Minimize token spend while maintaining quality	Reuse prompts, cache normalized data, limit context window

3.3. Out of Scope / Future Enhancements

Features identified as Grand Vision but NOT part of UMHackathon 2026 MVP:

01. In-App Real-Time Messaging
 - a. Direct chat between renter and landlord within Homie app
 - b. Deferred to Phase 3 (requires Telegram Bot API + message threading)
02. Promotional & Discount Management
 - a. Promo codes, landlord incentives, seasonal discounts
 - b. Requires partnership integrations; post-hackathon
03. Scheduled Delivery & Viewing Appointments
 - a. Ability to schedule property viewing times
 - b. Requires calendar integration + landlord availability API
04. Analytics & Reporting Dashboard
 - a. Search trends, popular areas, price heatmaps
 - b. Deferred to Phase 2
05. User Accounts & Saved Searches
 - a. Login system, saved filters, search history
 - b. Requires authentication service; lower priority for MVP
06. Landlord Portal
 - a. Dashboard for landlords to respond to inquiries, manage postings
 - b. Separate subsystem; out of scope for renter-focused MVP

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

Closed Testing: Deploy to internal staging environment

- QA team validates all 5 core features end-to-end
- Performance testing: Verify P95 latency $\leq 8s$, cost per session < 0.10 SGD
- Edge case testing: No landlord contact, network failures, token exhaustion

Canary Release : 5% of real users

- Monitor error rate, latency, token consumption
- Success Criteria: $<1\%$ error rate, P95 latency $\leq 10s$
- Rollback trigger: $>2\%$ errors OR cost/session > 0.15 SGD

A/B Testing: Compare two scoring weights

- Variant A: Emphasis on price (40%)
- Variant B: Emphasis on location (40%)
- Metric: User click-through rate to outreach
-

4.1.2. Emergency Rollback (ER) & Golden Release:

Scenario	Condition	Action
P1 Critical	<i>$>5\%$ scraper failures OR GLM API unavailable for >5 min</i>	<i>Automatic rollback to Golden Release (last stable version)</i>
P2 High	<i>Payment/Telegram integration failing</i>	<i>Fallback to manual contact info display</i>
P3 Medium	<i>Scoring calculation error</i>	<i>Use cached scores from the previous session</i>

Golden Release Management:

- Tagged in Git as golden-release-v1
- Docker image pushed to registry with immutable tag
- Database backup taken before each canary release
- Rollback command: `kubectl set image deployment/homie-backend homie-backend=homie:golden-release-v1`

4.1.3. Priority Matrix:

Priority	Impact	Trigger Condition	Action	Ownership
P1 Critical	System Down	Search API returning 5XX errors for >2 min	Immediate rollback to golden release; notify on-call engineer	DevOps
P1 Critical	Core Loss	GLM API consistently returning 401 (auth) errors	Check API key validity; swap to backup key if available	Backend Lead
P1 Critical	Data Loss	Database write failures on Listings table	Stop accepting new searches; restore from backup	DevOps + DB Admin
P2 High	Major Degradation	Search latency >15s (P95)	Increase Uvicorn worker count; reduce max_results per scraper	DevOps + Backend
P2 High	Quality Issue	Scraper success rate <70% on any source	Disable that scraper; alert engineering team for fix	Backend Lead

P2 High	Cost Overrun	Token consumption >12,000 per session (budget=10,000)	Reduce context window; compress prompts	AI/MLOps Lead
P3 Medium	User Friction	Outreach message generation >5s	Implement fallback template messages	Backend Lead

4.2. Monitoring:

Metric	Target	Tool	Alert Threshold
API Latency (P95)	$\leq 8s$	CloudRun / Datadog	> 15s for 5 min
Error Rate	< 1%	Sentry / CloudRun Logs	> 2% for 5 min
Scraper Success Rate	> 95% per source	Custom logging	< 80% for any source
Token Consumption	Avg 8000/session	Custom meter in GLM client	> 15000 per session
Database Connection Pool	< 50 active	SQLAlchemy pool stats	> 80 active connections
Telegram Delivery Rate	> 95%	Telegram Bot API callbacks	< 90% for 10 messages

Frontend Load Time	< 3s	Vercel Analytics / WebVitals	> 5s
---------------------------	------	---------------------------------	------

4.3. Assumptions: State key operational & environmental conditions that you assumed to be valid while development of system and deployment. [e.g.

a. Operational Assumptions:

- **User Connectivity:** Renters have stable internet (3G+) to load results in <8s
- **Landlord Contact:** Landlords actively use Telegram (high adoption in Malaysia)
- **Listing Data Freshness:** ibilik, iProperty, Facebook updates listings within 24 hours
- **Language Detection:** Listings in Malay, English, or Mandarin (majority of Malaysia market)
- **GLM API Reliability:** ZhipuAI maintains 99% uptime; occasional 401 errors are recoverable via retry

b. Environmental Assumptions:

- PostgreSQL (Neon) is available 24/7 with <100ms latency to backend
- Playwright browser instances can be launched in containerized environment
- Telegram Bot API endpoints are reachable from deployment region (no geo-blocking)
- No IP-based rate limiting from target scraping websites (moderate request rates expected)

4.4. External Dependencies: Do list all the external tools that are applied to your system to function. [e.g. below]

Tools	Purpose	Risks
GLM-5.1 API	Core reasoning engine for orchestration and extraction.	API rate limits or downtime may halt the core workflow.
Telegram Bot API	Send messages & deep links to users	API rate limits or downtime may halt the core workflow.

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:

No.	Activity	Day											
		0	1	2	3	4	5	6	7	8	9	10	
1.	Research & brainstorming												
2.	PRD Documentation												
3.	Backend MVP Implementation												
4.	Frontend MVP Implementation												
5.	Integration & Refinement												
6.	Testing & Optimization												
7.	SA Documentation												
8.	STA Documentation												
9.	Pitch Deck												
10.	Presentation & Video Editing												
11.	Checking & Submission												

- 5.2. **Team Members Role:** Highlight the role of team members [e.g. Person A – Role Backend Lead – Fast API Microservices & Schema Design]

Name	Roles
Choi Jin Seng	• Team Lead • Technical Lead(Backend & Frontend) • Responsible for Sample Testing Analysis Documentation(QTAD) • Presentation
Wong Zixin	• Responsible for System Analysis Documentation(SAD) • Presentation
Muhammad Afiq bin Shukur	• Responsible for Product Requirement Documentation(PRD) • Presentation
Ng Geok Liu	• Pitch Desk Slides • Presentation
Josephine Ding Jie Yu	• Video Editing • Presentation

5.3. **Recommendations:**

To ensure the long-term viability of Homie beyond the hackathon, the following technical recommendations are proposed:

- Scalability:** Implementation of **Redis Cache** for frequently searched locations (e.g., "Cheras" or "Kuala Lumpur") and listing data to reduce redundant third-party scraping and minimize API latency.
- Performance:** Integration of a message queue like **Celery or RabbitMQ** to handle a high volume of concurrent search sessions, allowing the orchestrator to scale horizontally.
- Reliability:** Development of **Semantic Deduplication Logic** using vector embeddings to identify the same rental unit listed across different platforms with different titles.
- AI Efficiency:** Implementation of **Prompt Caching** for repeating queries to stay within the defined session token budget and improve response times for summary reports.
- Resilience:** Enhancing the **Facebook Scraper** with a rotating session management system to mitigate account-level rate limiting or session expiration